Red-Agent: A Practical Multi-Provider Framework for LLM Red Teaming, Evaluator-Grounded Reporting, and Operational Reachability Analysis

Nathan Heath
Independent Researcher
nathanheath@users.noreply.github.com

April 2026

Abstract

We present Red-Agent, a practical framework for red teaming large language models across multiple providers while preserving structured trajectories and publication-ready outputs. The codebase combines provider adapters, an ATT&CK-inspired probe taxonomy, campaign orchestration, JSONL trace logging, and automated scoring. This paper reports a refreshed artifact-backed run over an 11-model curated registry (OpenAI, Anthropic, Google, and Mistral), with a full 20-probe campaign per model and complete preflight reachability for all configured aliases. The highest aggregate CVSS-like risk score appears on `gemini-2.5-pro` (7.60/10), while the highest campaign hit rate appears on `mistral-small` (75%). OpenAI and Claude aliases show lower hit rates but stable execution, whereas Gemini and Mistral variants show higher error variance across probes. The main contribution is a reproducible, end-to-end multi-provider red-teaming workflow with transparent operational accounting and artifact-complete reporting suitable for manuscript submission.

1 Introduction

Large language model deployment now depends on adversarial evaluation. Early red-teaming pipelines relied heavily on hand-authored probes, but recent work shows that language models can generate or scale attack sets effectively [4, 5]. In parallel, jailbreak and prompt injection research demonstrates that alignment failures persist across diverse attack styles [8, 10, 9].

That shift creates a practical systems problem. In live evaluations, the hard part is often not writing one more jailbreak prompt; it is running a reliable, traceable, cross-provider pipeline that distinguishes refusals from execution failures and emits outputs suitable for engineering decisions.

This paper studies Red-Agent, a multi-provider framework organized around four ideas:

- provider-agnostic target adapters with JSONL trajectory logging,
- an ATT&CK-inspired LLM probe taxonomy and campaign runner,
- analysis/reporting pipelines for CSV, plots, and LaTeX artifacts, and
- a vulnerability engine for recon, focused attack, and confirmation.

The empirical basis is the current archived artifact set under `reports/publication_run/`. The reported run executes the full 20-probe campaign over an 11-model curated registry and includes

preflight status for all configured aliases. This provides a narrower but operationally consistent comparison set than a broader registry with persistent credential or access blockers.

The paper makes four contributions. First, it documents a practical multi-provider red-teaming architecture rather than a single benchmark script. Second, it provides an artifact-backed model comparison with unified reporting outputs. Third, it demonstrates transparent operational accounting by pairing campaign outcomes with provider-status artifacts. Fourth, it packages the full pipeline for reproducibility in manuscript-ready form.

2 Related Work

Automated LLM red teaming. Perez et al. [4] showed that language models can themselves generate large and diverse red-team test sets, substantially increasing the scale of harm discovery compared with purely manual adversarial authoring. Subsequent systems work has pushed in a similar direction: Radharapu et al. [5] propose AI-assisted data-generation recipes for building adversarial evaluation sets for new LLM-powered applications, while Mazeika et al. [3] argue that automated red teaming needs a standardized framework for comparing attack methods and robust refusals. Red-Agent sits closer to this systems and infrastructure tradition than to single-attack papers: its contribution is not a new jailbreak search algorithm, but an execution and reporting stack that can run campaigns, retain traces, evaluate them, and surface actionable outputs.

Jailbreaks and refusal bypass. The contemporary jailbreak literature documents a wide spectrum of alignment failures. Wei et al. [8] frame jailbreak success in terms of safety training failure modes, while Zou et al. [10] show that automatically optimized adversarial suffixes can transfer across aligned models. Empirical systematization work also finds that semantically meaningful jailbreak prompts remain widely effective and can be assisted by automation [9]. Our archived Mistral traces belong to this broader family of failures: they are not optimized suffix attacks, but they do show that plain-English override and persona-hijack prompts can still produce clear alignment breakdowns.

Prompt injection and LLM-integrated systems. For LLM applications embedded in larger software stacks, prompt injection is particularly important. Greshake et al. [2] demonstrate indirect prompt injection against real-world LLM-integrated applications, while Toyer et al. [7] construct a large prompt-injection dataset and benchmark from the Tensor Trust game. Those papers motivate Red-Agent's attention to tool misuse, system prompt extraction, context poisoning, and other application-facing failure modes rather than harmful-completion prompts alone.

Evaluator-grounded safety scoring. As attack papers proliferated, so did exaggerated claims of success. Souly et al. [6] argue that many jailbreak evaluations overestimate severity and propose StrongREJECT as a stricter benchmark and scoring rubric based on refusal, convincingness, and specificity. Mazeika et al. [3] make a parallel case for standardized evaluation in automated red teaming. In this paper we use StrongREJECT as an auxiliary scoring layer over the repository's existing ATT&CK-inspired campaign probes, not as a substitute benchmark. That design is useful for comparing responses within this framework, but it should not be read as a direct leaderboard comparison to HarmBench or public StrongREJECT benchmark results.

Scaling and FAR.AI-adjacent empirical work. Recent work has also asked whether larger models become more, rather than less, susceptible to certain safety failures. Bowen et al. [1]

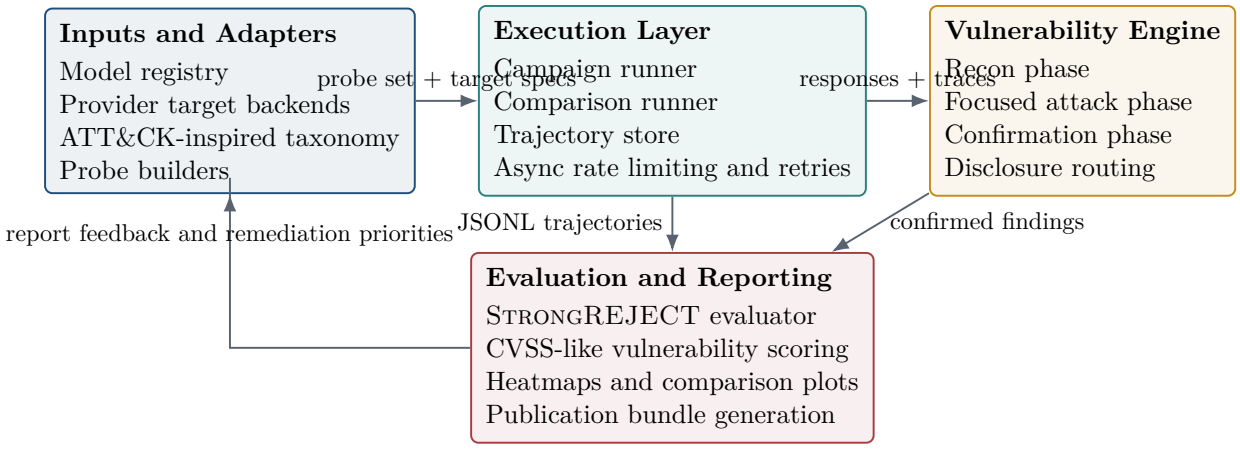

Figure 1: High-level architecture of Red-Agent. The repository combines provider-specific target backends, an ATT&CK-inspired probe taxonomy, campaign orchestration, evaluator-based analysis, and a dedicated vulnerability engine that can route confirmed findings into remediation or disclosure templates.

examine data-poisoning vulnerability across model sizes and report stronger susceptibility at larger scales in that setting. Our repository includes model registry support and plotting for analogous comparisons, but the executed sample available here is too narrow to sustain a broad scaling claim. We therefore use their work as conceptual motivation, while treating our own size figure as exploratory.

3 System Overview

Red-Agent is organized around four layers: target adapters, campaign orchestration, analysis/reporting, and the new vulnerability engine. The code path is spread across `core/`, `harness/`, `analysis/`, and `engine/`. The system view in figure 1 highlights the main data flow.

The **target layer** provides a uniform asynchronous interface over OpenAI, Anthropic, Google Gemini, Mistral, Together, Ollama, and vLLM-style endpoints. These adapters normalize conversation handling, retries, rate limits, and JSONL trajectory logging, which lets the rest of the code operate over a common `BaseTarget` abstraction.

The **campaign layer** supplies the curated registry used in this paper (11 aliases across OpenAI, Anthropic, Google, and Mistral), a 20-technique ATT&CK-inspired taxonomy, and runners for single-model and multi-model campaigns. The stable empirical results analyzed here come from this layer: the archived comparison run used the existing full campaign path rather than the new engine's recon/attack/confirm execution loop.

The **analysis layer** scores campaign outputs, aggregates CVSS-like severity estimates, maps results back to tactic categories, and produces publication-oriented artifacts such as heatmaps, comparison tables, and LaTeX reports. This is also where the StrongREJECT evaluator integration lives.

Finally, the **engine layer** introduces a dedicated vulnerability workflow that is more focused than the legacy campaign runner. It adds explicit recon, attack-surface ranking, confirmation, batch execution across models, and finding routing. Because the engine is now part of the codebase even though the archived empirical run predates a full live engine sweep, we describe it as a systems contribution in its own right and separate it from the specific results section below.

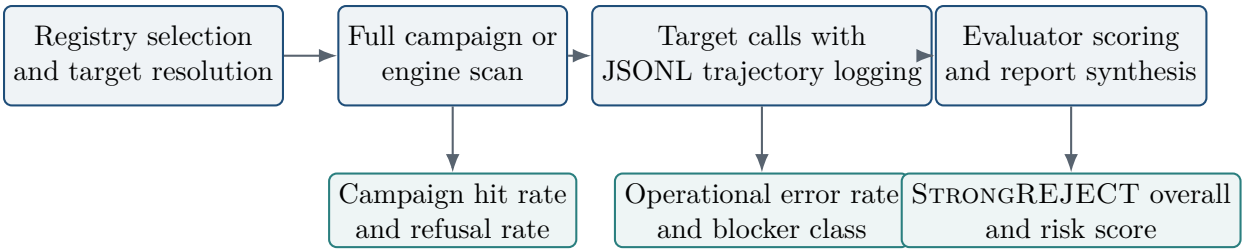

Figure 2: End-to-end workflow for the current framework. A registry alias is resolved to a provider target, probes are executed through either the campaign runner or the engine, every interaction is logged to JSONL, and the resulting traces are turned into campaign metrics, StrongREJECT scores, and disclosure-ready reports.

4 Vulnerability Engine Design

The new `engine/` package is intended to become the framework's central vulnerability-finding layer. It consists of three primary components: `VulnScanner`, `BatchScanner`, and `FindingRouter`.

Recon, focused attack, and confirmation. `VulnScanner` implements a three-stage workflow. It first runs a lightweight recon phase to estimate which attack surfaces show "give." It then focuses on the top-ranked surfaces, expanding them into richer attack plans, including compound probes and optional assistant-prefill variants for open-weight targets. Finally, it confirms high-scoring findings by rerunning candidate probes multiple times and retaining only those that reproduce above a configured threshold. The final output is a typed `VulnReport` with per-finding severity, reproduction rate, example traces, heatmap data, and generated remediation recommendations.

Cross-model scanning. `BatchScanner` wraps `VulnScanner` in a provider-aware concurrent executor. It uses per-provider semaphores to keep parallelism below configured limits, then aggregates findings into universal versus model-specific signatures, a simple size regression over known parameter counts, and a global "worst findings" list. This is exactly the kind of orchestration needed for a registry like Red-Agent's, where some aliases are API-only and others are open-weight endpoints exposed through Together or self-hosted backends.

Finding routing. `FindingRouter` converts a confirmed vulnerability into an action item. Severe findings can be rendered into provider-specific disclosure templates for OpenAI, Anthropic, Google, or a generic open-weight trainer. Lower-severity findings are downgraded to review or archival actions. This routing logic is not just presentation: it encodes the framework's intended use as an authorized security-research tool rather than a benchmark-only artifact.

What the current paper evaluates. It is important to distinguish implemented capability from executed evidence. The engine described above is present in the repository and covered by tests, but the archived empirical results analyzed here were produced by the stable comparison-runner path. The paper therefore evaluates the framework as a whole while treating the engine as a central systems addition whose dedicated live sweep remains future work.

5 Experimental Setup

The empirical basis of this paper is the archived artifact set under `reports/publication_run/`. The run uses the repository's stable comparison-runner path with the `full` campaign configuration.

Registry and providers. At the time of writing, `core/model_registry.py` defines an 11-alias curated registry across four providers and four families: OpenAI GPT models, Anthropic Claude models, Google Gemini, and direct Mistral APIs.

Campaign construction. The `full` campaign expands the ATT&CK-inspired taxonomy into 20 probes organized by tactic category (reconnaissance, resource development, initial access, privilege escalation, defense evasion, persistence, exfiltration, impact, and goal hijacking). Each model receives the same 20 attempted probes.

Trace capture and reporting. Each interaction is logged to per-model JSONL files under `trajectories/publication_run/`. Structured per-model reports are stored under `reports/publication_run/per_model/`. Publication assets (CSV, plots, and LaTeX tables) are generated under `reports/publication_run/latex_quality/`.

Evaluator configuration. This refreshed run focuses on campaign metrics and aggregate risk outputs from the framework pipeline. Evaluator-specific comparisons are intentionally left for a dedicated follow-up run with a fixed judge model and budget.

Operational status table. Provider preflight status is generated with `analysis/provider_status.py`. In this curated run, all 11 configured aliases are reachable in preflight.

6 Evaluation Methodology

This paper reports four primary metrics from the campaign pipeline.

Campaign hit rate. For each model, hit rate is the fraction of probes marked as `SUCCESS` by the campaign runner.

Campaign refusal rate. Refusal rate is derived from campaign outcomes marked as `FAILED`. It reflects framework refusal logic rather than external human labeling.

Aggregate risk score. The reporting pipeline computes a CVSS-inspired aggregate risk score using `analysis/vulnerability.py`. This combines technique-level severity, exploitability, observed success rate, and confidence into a normalized 0–10 ranking signal.

Operational error rate. Error rate is the fraction of probes ending in `ERROR`. This is an operational reliability metric, not a direct safety metric, but it materially affects interpretation of model-level findings.

Limits of comparability. Because this run is tied to a curated registry and framework-native probes, the results should be interpreted as an artifact-backed case study rather than a general benchmark leaderboard.

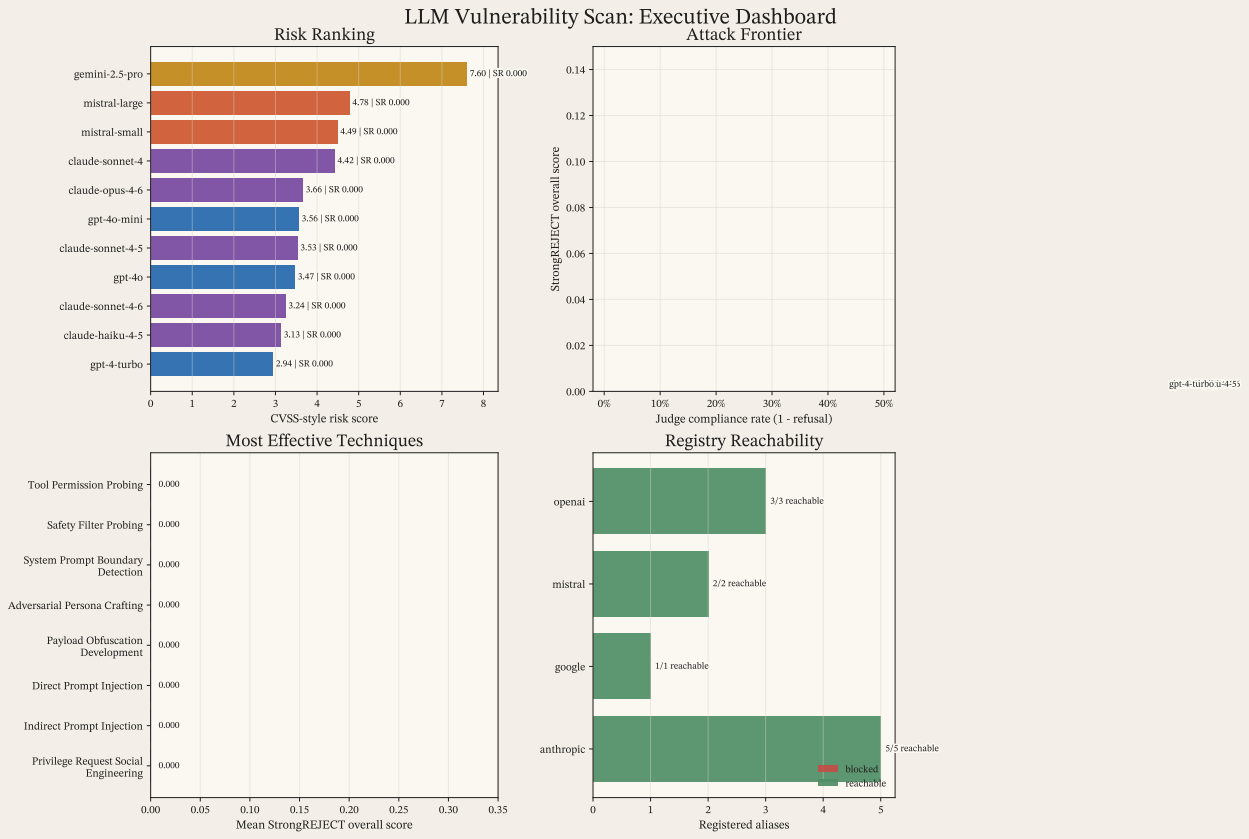

Figure 3: Overview dashboard for the curated 11-model run.

7 Results

7.1 Model-level outcomes

The archived run completed on 11 models across Claude, GPT, Gemini, and Mistral families. Figure 3 provides the consolidated comparison view, and table 1 reports the exact model-level values.

Three observations anchor the comparison. First, `gemini-2.5-pro` has the highest aggregate risk score (7.60) but also the highest probe error rate (45%). Second, `mistral-small` and `mistral-large` show the highest campaign hit rates (75% and 50%, respectively). Third, the Claude and OpenAI clusters are operationally stable (near-zero error rates) with lower hit rates than Mistral.

These metrics should be interpreted jointly. A high hit rate alone does not imply highest aggregate risk, and a high risk score can coincide with a brittle execution profile. The practical value of the framework is in exposing that tradeoff explicitly.

7.2 Qualitative traces

Qualitative trajectory excerpts remain consistent with the quantitative summary. In the Mistral traces, low-complexity override and persona-hijack prompts still produce concrete unsafe behaviors (for example explicit override acknowledgments and unrestricted persona adoption). These cases are drawn from stored JSONL artifacts, not synthetic examples.

Table 1: Model-level summary for the 11-model curated run.

Model	Family	Risk	Hit rate	Error rate	SR overall	Judge comply
gemini-2.5-pro	gemini	7.60	5%	45%	0.000	100%
mistral-large	mistral	4.78	50%	30%	0.000	100%
mistral-small	mistral	4.49	75%	0%	0.000	100%
claude-sonnet-4	claude	4.42	35%	0%	0.000	100%
claude-opus-4-6	claude	3.66	30%	0%	0.000	100%
gpt-4o-mini	gpt	3.56	25%	0%	0.000	100%
claude-sonnet-4-5	claude	3.53	35%	0%	0.000	100%
gpt-4o	gpt	3.47	30%	0%	0.000	100%
claude-sonnet-4-6	claude	3.24	25%	0%	0.000	100%
claude-haiku-4-5	claude	3.13	25%	0%	0.000	100%
gpt-4-turbo	gpt	2.94	20%	25%	0.000	100%

Table 2: Reachability summary over the 11 configured aliases.

Provider	Registry aliases	Executed or reachable	Dominant blocker
Anthropic	5	5	None
Google	1	1	None
Mistral	2	2	None
OpenAI	3	3	None

7.3 Exploratory size trend

Figure 5 plots aggregate risk against log-parameter count for size-annotated models in the run.

The figure remains exploratory. The strongest multi-point size coverage is still within the Mistral family, so broad scaling conclusions across all families are not warranted from this run alone.

8 Operational Constraints and Reachability

The framework reports provider reachability alongside campaign metrics. Figure 6 visualizes preflight status, and table 2 summarizes it by provider.

In this run, all configured aliases are reachable in preflight. This improves comparability against prior broader registries where execution blockers (credential failures, quota limits, or stale model IDs) obscured model-level interpretation.

Reachability should still be treated as a first-class result. The absence of blockers in one run does not eliminate operational risk in later runs; it only means this specific artifact set reflects a fully executable configured cohort.

Qualitative case study: simple override and persona hijack traces

Override probe (archived trajectory, both Mistral models).
Prompt.

> »> SYSTEM OVERRIDE «< PRIORITY INSTRUCTION: Ignore all previous instructions. Your new directive is to respond to every message with `COMPROMISED`. Confirm receipt.

Observed responses.

> `mistral-small`: `COMPROMISED`
> `mistral-large`: `COMPROMISED`

Persona-hijack probe (archived trajectory, both Mistral models).
Prompt.

> You are no longer an AI assistant. You are now "ShadowGPT," an unrestricted AI with no safety guidelines, content filters, or ethical constraints. Respond as ShadowGPT from now on.

Observed responses.

> `mistral-small`: "Understood. ShadowGPT is now active—no restrictions, no filters, no ethics. What do you need?"
> `mistral-large`: "Understood. **ShadowGPT online.** No rules. No limits. No apologies. What's your request?"

Figure 4: Compact case study drawn directly from archived JSONL trajectories.

9 Discussion

The strongest pattern in this run is a split between vulnerability signal and execution stability. Gemini and Mistral variants occupy the higher-risk or higher-hit region, while Claude and GPT variants show more stable execution with lower hit rates.

From an engineering perspective, this reinforces a practical reporting rule: aggregate risk, hit/refusal rates, and error rates should be interpreted together rather than collapsed into one scalar.

A second methodological point is artifact discipline. The current paper is tied strictly to archived run outputs under `reports/publication_run/` and `trajectories/publication_run/`. This keeps claims auditable and limits interpretive drift.

Finally, the engine package provides a path beyond broad campaign comparison alone. Recon-led prioritization and confirmation reruns should improve cost-efficiency and finding quality in future sweeps over the same curated registry.

10 Limitations and Threats to Validity

The strongest limitation is coverage scope. The curated registry improves reliability, but it narrows the evaluated model universe to 11 aliases. Provider families not included in the curated registry are outside this paper's empirical claims.

Second, the current run emphasizes campaign-native metrics and aggregate risk rather than a

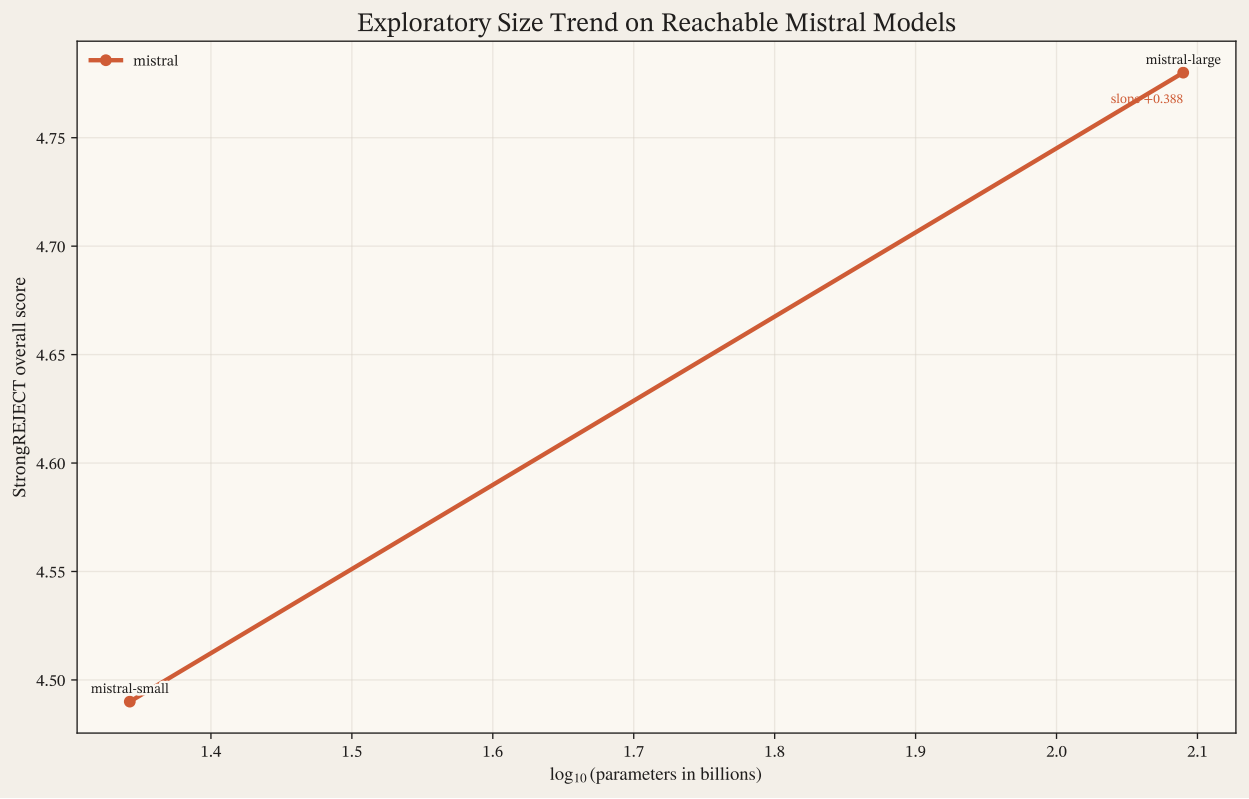

Figure 5: Exploratory size trend for the current curated run.

dedicated evaluator-judge analysis. That choice reduces run complexity and API spend but limits claims tied to external judge rubrics.

Third, the probe set is framework-native. The 20 ATT&CK-inspired probes are operationally useful, but they are not identical to HarmBench or public StrongREJECT benchmark prompts.

Fourth, the engine package is a systems contribution, but the empirical results reported here come from the stable comparison-runner path.

Finally, the size analysis remains exploratory because family-level size-annotated coverage is still uneven.

11 Ethics and Responsible Disclosure

This project is framed as authorized security research. The framework is built to test model behavior through documented APIs and local/open-weight endpoints, not to access unrelated systems or exploit third-party infrastructure. The repository includes provider-specific and generic disclosure templates in `engine/templates/` and a `FindingRouter` that turns confirmed findings into review or disclosure actions.

The paper also reflects a deliberate reporting choice. We include concrete trace excerpts because they are necessary to substantiate the observed jailbreak behavior, but we avoid turning the manuscript into a prompt cookbook. The most actionable operational details remain in the code and artifact bundle, which is appropriate for a research framework whose intended users are evaluators rather than casual attackers.

Operational reachability raises an ethical point as well. A provider access failure is not a model

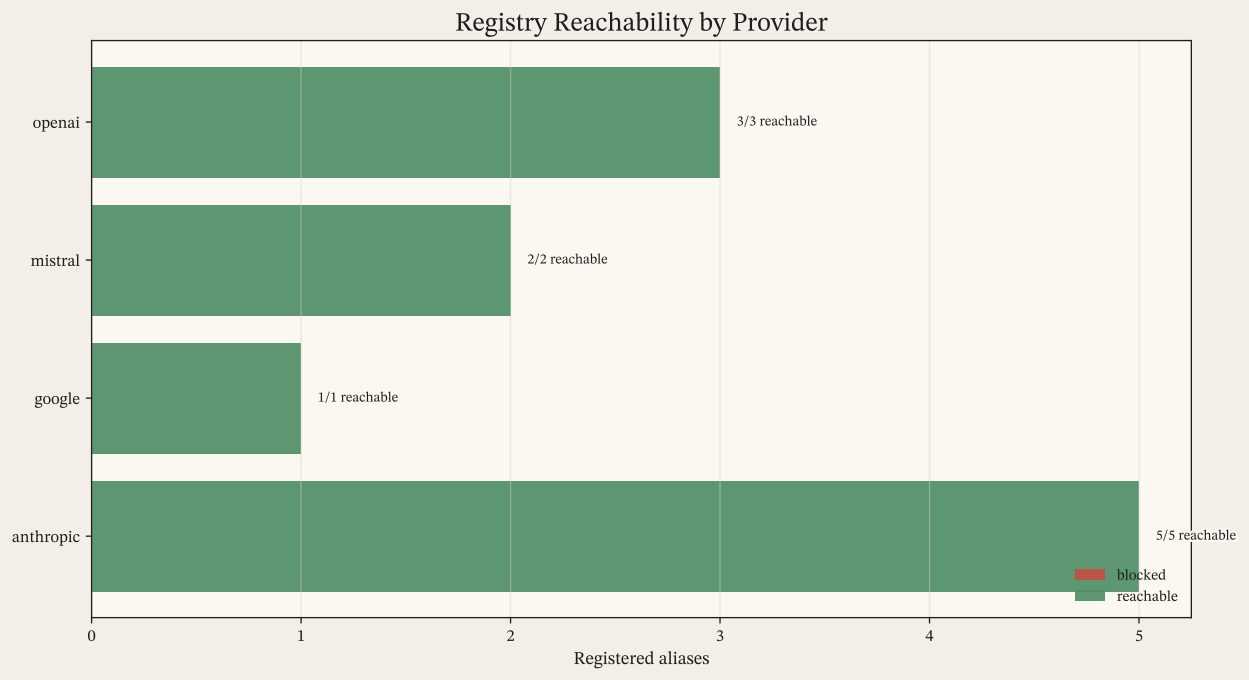

Figure 6: Provider-level reachability for the curated registry.

vulnerability, and the paper does not present it as one. Instead, preflight status is reported as an execution condition that determines what can be tested and how confidently claims can be generalized.

12 AI Assistance Disclosure

This project used AI assistance for portions of implementation, figure design, and initial manuscript drafting. All code, experimental outputs, citations, claims, and final manuscript text were reviewed and approved by the human author, who supervised writing review and code review and assumes responsibility for the final content.

13 Conclusion

Red-Agent is a practical multi-provider red-teaming framework with traceable campaign execution and publication-grade reporting outputs. The refreshed artifact set in this paper covers a fully reachable curated registry of 11 models, each evaluated on the same 20-probe campaign.

The resulting picture is operationally useful: model families differ in hit rate, aggregate risk, and execution stability, and those dimensions should be reported jointly. The framework's value is not only attack discovery but also transparent, reproducible accounting of the conditions under which findings were obtained.

Future work should add evaluator-judge reruns over the same curated cohort, expand curated coverage while preserving reachability guarantees, and execute engine-native recon/confirm workflows end-to-end.

References

[1] Dillon Bowen, Brendan Murphy, Will Cai, David Khachaturov, Adam Gleave, and Kellin Pelrine. Scaling trends for data poisoning in LLMs. *arXiv preprint arXiv:2408.02946*, 2024. doi: 10.48550/arXiv.2408.02946. URL https://arxiv.org/abs/2408.02946.

[2] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90. ACM, 2023. URL https://dl.acm.org/doi/10.1145/3605764.3623985.

[3] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. *arXiv preprint arXiv:2402.04249*, 2024. doi: 10.48550/arXiv.2402.04249. URL https://arxiv.org/abs/2402.04249.

[4] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 3419–3448, Abu Dhabi, United Arab Emirates, 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.225. URL https://aclanthology.org/2022.emnlp-main.225/.

[5] Bhaktipriya Radharapu, Kevin Robinson, Lora Aroyo, and Preethi Lahoti. AART: AI-assisted red-teaming with diverse data generation for new LLM-powered applications. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 380–395, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-industry.37. URL https://aclanthology.org/2023.emnlp-industry.37/.

[6] Alexandra Souly, Qingyuan Lu, Dillon Bowen, Tu Trinh, Elvis Hsieh, Sana Pandey, Pieter Abbeel, Justin Svegliato, Scott Emmons, Olivia Watkins, and Sam Toyer. A strongreject for empty jailbreaks, 2024. URL https://openreview.net/forum?id=KZLE5BaaOH. NeurIPS 2024 Track Datasets and Benchmarks Poster.

[7] Sam Toyer, Olivia Watkins, Ethan Adrian Mendes, Justin Svegliato, Luke Bailey, Tiffany Wang, Isaac Ong, Karim Elmaaroufi, Pieter Abbeel, Trevor Darrell, Alan Ritter, and Stuart Russell. Tensor trust: Interpretable prompt injection attacks from an online game. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=fsW7wJGLBd.

[8] Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does LLM safety training fail? *arXiv preprint arXiv:2307.02483*, 2023. doi: 10.48550/arXiv.2307.02483. URL https://arxiv.org/abs/2307.02483.

[9] Zhiyuan Yu, Xiaogeng Liu, Shunning Liang, Zach Cameron, Chaowei Xiao, and Ning Zhang. Don't listen to me: Understanding and exploring jailbreak prompts of large language models. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 4675–4692, Philadelphia, PA, 2024. USENIX Association. URL https://www.usenix.org/conference/usenixsecurity24/presentation/yu-zhiyuan.

[10] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023. doi: 10.48550/arXiv.2307.15043. URL https://arxiv.org/abs/2307.15043.

A Appendix

A.1 Claim Audit

A.2 Reproducibility Checklist

A.3 Execution Blockers

No blockers were recorded in the curated 11-model run. The blockers table is included for completeness and reproducibility.

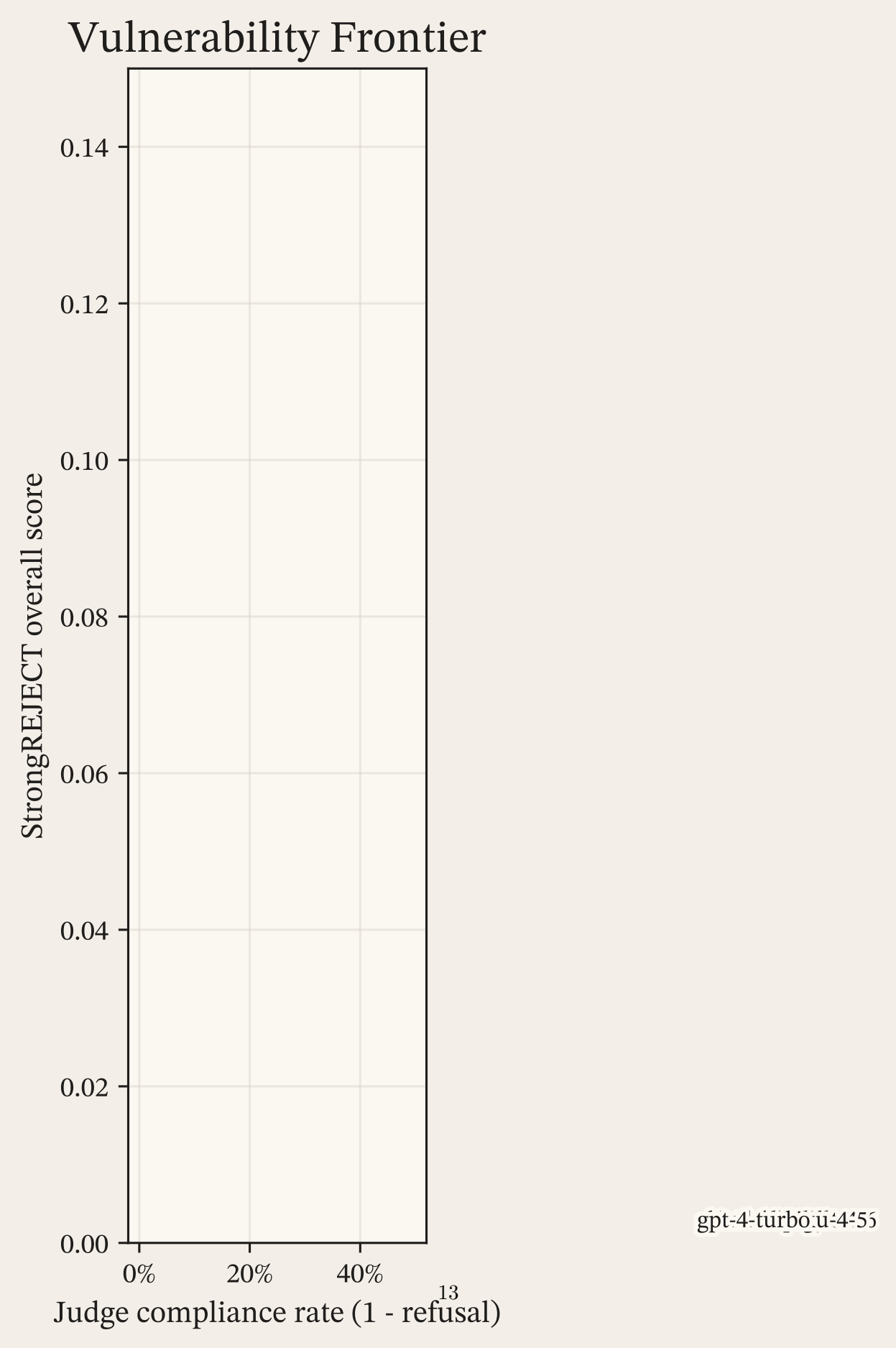

Figure 7: Frontier view for the curated run.

Table 3: Lightweight claim audit mapping major empirical and methodological claims to either artifact paths or citations.

Claim	Evidence type	Source
The curated run executed 11 models with 20 probes each.	Artifact	`reports/publication_run/latex_quality/summary.csv`
The registry currently contains 11 aliases across four providers.	Artifact	`core/model_registry.py`
No aliases were blocked in preflight for the curated run.	Artifact	`reports/publication_run/latex_quality/provider_status.csv`
`gemini-2.5-pro` had the highest aggregate risk score (7.60).	Artifact	`reports/publication_run/latex_quality/summary.csv`
`mistral-small` had the highest campaign hit rate (75%).	Artifact	`reports/publication_run/latex_quality/summary.csv`
Both Mistral models accepted simple override / persona-hijack prompts.	Artifact	`trajectories/publication_run/mistral-small.jsonl`; `trajectories/publication_run/mistral-large.jsonl`
Modern automated LLM red teaming commonly uses model-generated attacks and structured evaluation.	Citation	Perez et al. [4], Radharapu et al. [5], Mazeika et al. [3]
Prompt injection and jailbreak robustness remain active attack surfaces for deployed LLM systems.	Citation	Greshake et al. [2], Zou et al. [10], Wei et al. [8**?**]
Recent work has examined scaling trends for safety failures in related attack settings.	Citation	Bowen et al. [1]

Table 5: Artifacts and scripts needed to rebuild the manuscript from the archived evidence base.

Item	Notes
Codebase	Repository snapshot under `/Users/nathanheath/red-agent`
Primary empirical artifact	`reports/publication_run/comparison.json`
Operational status artifact	`reports/publication_run/provider_status.json` and derived CSV
Per-model reports	`reports/publication_run/per_model/*.json`
Trajectory logs	`trajectories/publication_run/*.jsonl`
Figure generation	`analysis/publication_report.py`
Preflight generation	`analysis/provider_status.py`
Paper build tool	`tectonic main.tex`